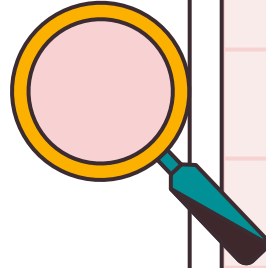
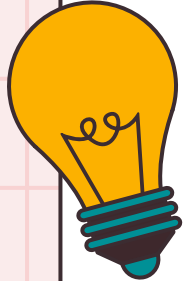
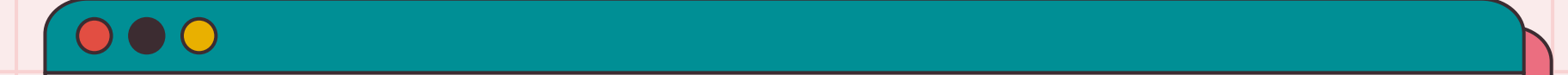




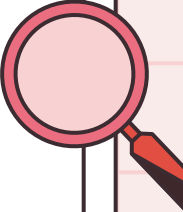
SpaceX Falcon 9 Landing Prediction

Muhamad Rizal





Executive Summary



Project Background: SpaceX has revolutionized the space industry by offering Falcon 9 launches at approximately \$62 million, significantly lower than the \$165 million charged by other providers. This cost efficiency is primarily achieved by reusing the rocket's first stage.

Project Scenario: Acting as a Data Scientist for a startup competitor, **Space Y**, the goal is to determine launch prices that can compete with SpaceX's pricing model.

Core Problem: Predicting whether the first stage will land successfully is critical, as a successful landing directly impacts the cost and feasibility of reusing the hardware.

<https://github.com/pyrall0712/SpaceX-Data-Science-Capstone/blob/main/README.md>

Methodology:

- **Data Acquisition:** Extracting data using the **SpaceX API** and performing **Web Scraping** to gather launch information.
- **Data Preparation:** Performing **Data Wrangling** to clean the dataset and create classification labels for landing success.
- **Exploratory Analysis:** Analyzing the data using **SQL** queries and **Pandas/Matplotlib** for trend visualization.
- **Interactive Visuals:** Building interactive maps with **Folium** and dynamic dashboards with **Plotly Dash**.
- **Predictive Analytics:** Training and evaluating **Classification Models** to predict the success of the first-stage landing.

Data Collection - SpaceX API

Objective: Retrieve historical launch data from the SpaceX API to analyze landing success factors.

(Request API -> Parse JSON -> Filter Falcon 9 -> Replace Missing Values).

Methodology:

- Sent HTTP GET requests to the SpaceX API endpoint (https://api.spacexdata.com/v4/launches/past) using the Python Requests library.
- Decoded the JSON response and converted it into a structured Pandas DataFrame using `json_normalize()`.

Data Filtering & Cleaning:

- Filtered the dataset to include only **Falcon 9** launches by removing Falcon 1 records.
- Handled missing values in the PayloadMass column by replacing them with the calculated mean value.

Final Output: Exported the processed data as `dataset_part_1.csv` for the next stage of analysis.

```
[12]: # Use json_normalize method to convert the json result into a dataframe
data = pd.json_normalize(response.json())
```

Using the dataframe `data` print the first 5 rows

```
[13]: # Get the head of the dataframe
data.head()
```

```
[13]: static_fire_date_utc  static_fire_date_unix    tbd   net  window          rocket  success  details  crew  ships  capsules          payloads          launchpad  auto_update  failures  flight_number  name          date_utc  date_unix          date_local  date_precision  upcoming          cores

0      2006-03-17T00:00:00.000Z      1.142554e+09  False  False    0.0  5e9d0d95da69955f709d1eb  False  Engine failure at 33 seconds and loss of vehicle  []  []  []  [5e9d0e4b5b6c3bb0005eeb1e1]  5e9d4502f500995a5e566b6  True  'reason': 'merlin engine failure']  1  FalconSat  2006-03-24T22:30:00.000Z  1143239400  2006-03-25T10:30:00+1200  hour  False  [[{'core': '5e9e289df5918033d3b26b27', 'flight': 1, 'grifflins': False, 'legs': False, 'reused': False, 'landing_attempt': False, 'landing_success': None, 'landing_type': None, 'landpad': None}]]
```

- Used helper functions (e.g., `getBoosterVersion`, `getLaunchSite`) to extract detailed information from IDs for rockets, payloads, launchpads, and cores.

https://github.com/pyrall0712/SpaceX-Data-Science-Capstone/blob/main/notebooks/jupyter-labs-spacex-data-collection-api.ipynb?short_path=a63d87e

Data Collection - Web Scraping

Objective: Extract Falcon 9 launch records from Wikipedia to supplement the dataset using a web scraping approach.

(Request URLParse with -> BeautifulSoup -> Extract Table Headers -> Parse Rows into Dictionary -> Convert to DataFrame)

Methodology:

- Performed an **HTTP GET** request to fetch the HTML content of the Wikipedia page titled "List of Falcon 9 and Falcon Heavy launches".

Print the page title to verify if the `BeautifulSoup` object was created properly

```
[8]: # Create a BeautifulSoup object from the response text content
soup = BeautifulSoup(response.text, 'html.parser')

# Print the page title to verify
print(soup.title)

<title>List of Falcon 9 and Falcon Heavy launches - Wikipedia</title>
```

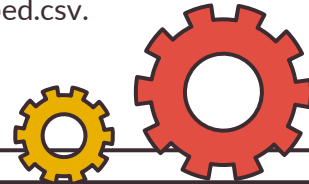
- Utilized the **BeautifulSoup** library to parse the HTML and create a soup object for data extraction.

<https://github.com/pyrall0712/SpaceX-Data-Science-Capstone/blob/main/notebooks/jupyter-labs-webscraping.ipynb>

Process:

- Extracted column names from HTML table headers (<th> elements) using a custom helper function `extract_column_from_header()`.
- Iterated through the HTML tables and rows (<tr>) to collect data for each launch, including flight numbers, launch sites, payload mass, and landing outcomes.
- Handled data cleaning during the scraping process to remove references, inconsistent formatting, and noise from the HTML tags.

Final Output: Parsed the collected dictionary into a **Pandas DataFrame** and exported the results to `spacex_web_scraped.csv`.



Data Wrangling and Feature Engineering

Objective: Process the raw SpaceX dataset to identify patterns and create binary labels for machine learning training.

(Load Data -> Count Launch Sites/Orbits -> Define Success/Failure -> Create Binary Class Column)

Exploratory Data Analysis (EDA):

- Calculated the number of launches for each launch site (e.g., CCAFS SLC 40, KSC LC 39A, and VAFB SLC 4E) using the `value_counts()` method.
- Analyzed the occurrence of various orbit types, such as LEO, GTO, ISS, and PO, to understand mission distribution.
- Identified the landing outcomes, categorizing successful missions (e.g., True ASDS, True RTLs) and unsuccessful missions (e.g., False Ocean, None None).

Feature Engineering:

- Defined a set of **bad outcomes** where the first stage did not land successfully.
- Created a new classification column named **Class**, assigning a value of **1** for successful landings and **0** for unsuccessful landings.

Result: Determined a preliminary success rate of approximately **66%** across the processed launch data.

<https://github.com/pyrall0712/SpaceX-Data-Science-Capstone/blob/main/notebooks/labs-jupyter-spacex-Data%20wrangling.ipynb>

```
# Assign it back to the dataframe
df['Class'] = landing_class

# Verify the first few rows
print(df[['Outcome', 'Class']].head(8))

# Calculate the success rate
success_rate = df['Class'].mean()
print(f"Success Rate: {success_rate}")
```

	Outcome	Class
0	None None	0
1	None None	0
2	None None	0
3	False Ocean	0
4	None None	0
5	None None	0
6	True Ocean	1
7	True Ocean	1

Success Rate: 0.6666666666666666

Exploratory Data Analysis (EDA) using SQL

Objective: Use SQL queries to extract specific insights and perform a detailed analysis of the SpaceX dataset
(Connect to DB -> Load CSV to SQL Table -> Execute Queries -> Extract Insightful Summaries)

```
%sql SELECT "Landing_Outcome", COUNT(*) AS Outcome_Count FROM SPACEXTABLE WHERE "Date" BETWEEN '2010-06-04' AND '2017-03-20' GROUP BY "Landing_Outcome" ORDER BY Outcome_Count DESC;
```

```
* sqlite:///my_data1.db  
Done.
```

Landing_Outcome	Outcome_Count
No attempt	10
Success (drone ship)	5
Failure (drone ship)	5
Success (ground pad)	3
Controlled (ocean)	3
Uncontrolled (ocean)	2
Failure (parachute)	2
Precluded (drone ship)	1

Database Setup:

- Loaded the SpaceX dataset into a **SQLite** database using the ipython-sql extension.
- Performed data cleaning to ensure accuracy, including the removal of blank rows.

Key SQL Insights:

- **Launch Site Analysis:** Identified unique launch sites (e.g., CCAFS LC-40, KSC LC-39A, VAFB SLC-4E).
- **Payload Metrics:** Calculated the total payload mass carried by boosters for specific customers like NASA (CRS).
- **Landing Milestones:** Successfully retrieved the date of the first successful ground pad landing (2015-12-22).
- **Performance Ranking:** Ranked landing outcomes (e.g., Success, Failure, No Attempt) by frequency to determine overall mission reliability.

Result: SQL analysis provided granular data on booster versions and mission parameters that were critical for predictive modeling.

https://github.com/pyrall0712/SpaceX-Data-Science-Capstone/blob/main/notebooks/jupyter-labs-eda-sql-coursera_sqlite.ipynb

Exploratory Data Analysis (EDA) with Visualization

Objective: Visualize relationships between various features and the launch outcome to identify patterns for prediction.

```
# TASK 7: Create dummy variables to categorical columns
features_one_hot = pd.get_dummies(features, columns=['Orbit', 'LaunchSite', 'LandingPad', 'Serial'])
```

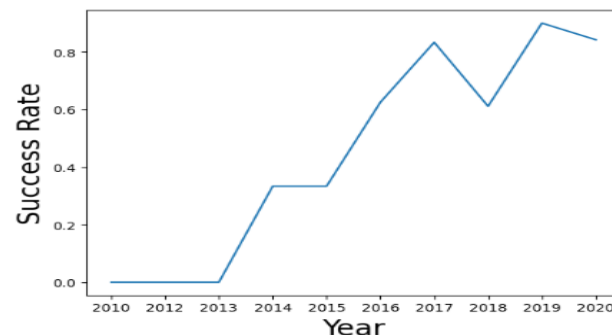
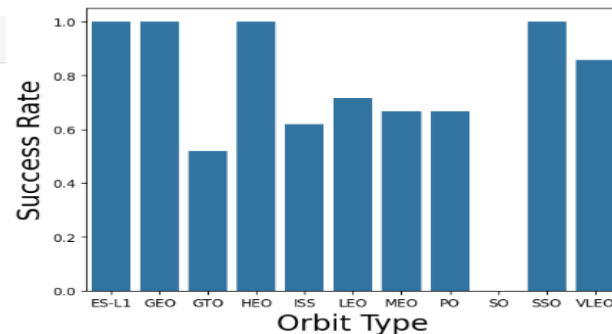
Key Visual Insights:

- **Flight Number vs. Payload Mass:** Observed that as the flight number increases, the first stage is more likely to land successfully, regardless of payload mass.
- **Success Rate by Orbit Type:** High success rates were identified for orbits like **ES-L1**, **GEO**, **HEO**, and **SSO**, while **GTO** showed a more mixed outcome.
- **Flight Number vs. Orbit:** In **LEO** orbit, success appears related to the number of flights; conversely, in **GTO** orbit, there is no strong relationship.
- **Yearly Launch Trend:** A line chart analysis revealed a significant positive trend in the landing success rate starting from 2013 and peaking in 2020.

Feature Engineering:

- Selected key features (PayloadMass, Orbit, LaunchSite, etc.) and applied **One-Hot Encoding** using `get_dummies` for categorical variables.
- Casted all numeric columns to **float64** to prepare the final dataset for machine learning models.

<https://github.com/pyrall0712/SpaceX-Data-Science-Capstone/blob/main/notebooks/edadataviz.ipynb>



Interactive Visual Analytics with Folium

Objective: Perform geographical analysis of launch sites to discover patterns related to location and proximity.

Key Tasks & Methodology:

- **Launch Site Mapping:** Marked all major SpaceX launch sites (CCAFS LC-40, KSC LC-39A, VAFB SLC-4E) on an interactive world map using folium.map and folium.Circle.
- **Success/Failure Markers:** Enhanced the map by adding MarkerCluster objects, using green markers for successful landings (class=1) and red markers for failed attempts (class=0).
- **Proximity Analysis:** Calculated distances from launch sites to critical infrastructures (coastlines, railways, highways, and cities) using the Haversine formula.

Geographical Findings:

- All launch sites are located in close proximity to the **coastline**, facilitating safer landing trajectories over the ocean.
- Launch sites are strategically positioned near **highways and railways** for transport logistics while maintaining a safe distance from major **population centers (cities)**.



https://github.com/pyrall0712/SpaceX-Data-Science-Capstone/blob/main/notebooks/lab_jupyter_launch_site_location.ipynb

Interactive Dashboard with Plotly Dash

Objective: Create a comprehensive web dashboard to analyze launch records and identify correlations between payload mass and landing success.

Key Dashboard Components:

- Dropdown Selector:** Implemented an interactive dropdown with options for each specific launch site (e.g., KSC LC-39A, VAFB SLC-4E) and an "All Sites" view.
- Range Slider:** Added a functional slider to filter launches based on **Payload Mass (kg)** from 0 to 10,000 kg.

Interactive Visualizations:

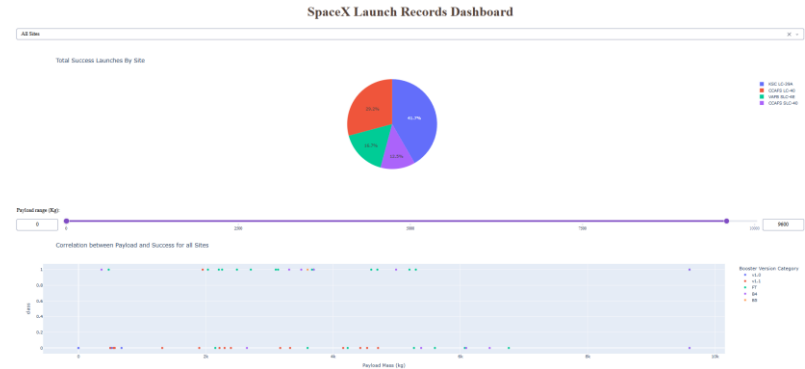
- Success Pie Chart:** Developed using `px.pie`. When "All Sites" is selected, it shows the success proportion per site. When a specific site is selected, it displays the ratio of success vs. failure.
- Correlation Scatter Plot:** Developed using `px.scatter`. Visualizes the relationship between **Payload Mass** and **Success**, color-coded by **Booster Version Category** for granular analysis.

Technical Implementation:

- Utilized **Dash Callbacks** to ensure the charts update dynamically based on user input from the dropdown and slider.

```
# TASK 4: Callback function for "site-dropdown" and "payload-slider" as inputs
@app.callback(Output(component_id='success-payload-scatter-chart', component_property='figure'),
              [Input(component_id='site-dropdown', component_property='value'),
               Input(component_id='payload-slider', component_property='value')])
def get_scatter_chart(entered_site, payload_range):
    low, high = payload_range
    mask = (spacex_df['Payload Mass (kg)'] >= low) & (spacex_df['Payload Mass (kg)'] <= high)
    filtered_df = spacex_df[mask]

    if entered_site == 'ALL':
        fig = px.scatter(filtered_df, x='Payload Mass (kg)', y='class',
                        color='Booster Version Category',
                        title='Correlation between Payload and Success for all Sites')
        return fig
    else:
        filtered_df = filtered_df[filtered_df['Launch Site'] == entered_site]
        fig = px.scatter(filtered_df, x='Payload Mass (kg)', y='class',
                        color='Booster Version Category',
                        title=f'Correlation between Payload and Success for site {entered_site}')
        return fig
```



<https://github.com/pyrall0712/SpaceX-Data-Science-Capstone/blob/main/scripts/spacex-dash-app.py>

Interactive Dashboard with Plotly Dash

Objective: Build a machine learning pipeline to predict the success of the first-stage landing for Falcon 9 rockets.

Methodology:

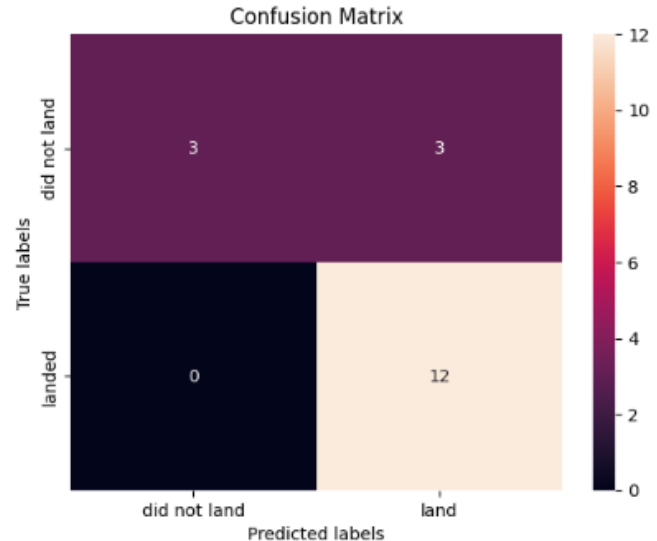
- **Data Preparation:** Standardized the feature data using StandardScaler to ensure all features are on the same scale for optimal model performance.
- **Data Splitting:** Split the dataset into training and test sets (20% for testing, resulting in 18 test samples) using train_test_split.
- **Model Training & Tuning:** Utilized GridSearchCV with 10-fold cross-validation to find the best hyperparameters for four classification algorithms: **Logistic Regression**, **SVM**, **Decision Tree**, and **KNN**.

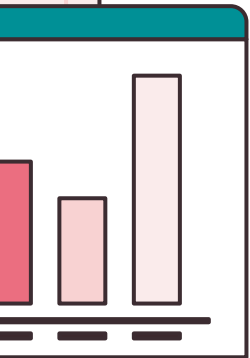
Performance Evaluation:

- **Logistic Regression:** Achieved a test accuracy of **83.33%** with tuned parameters.
- **Support Vector Machine (SVM):** Achieved a test accuracy of **83.33%**.
- **K-Nearest Neighbors (KNN):** Achieved a test accuracy of **83.33%**.
- **Decision Tree:** Achieved a test accuracy of **77.78%**.

Conclusion: While all models performed similarly on the small test set, the **Decision Tree** emerged as the best performing method during the training phase with an accuracy of **90.54%**.

https://github.com/pyrall0712/SpaceX-Data-Science-Capstone/blob/main/notebooks/SpaceX_Machine%20Learning%20Prediction_Part_5.ipynb





Thanks!

CREDITS: This presentation template was created by [Slidesgo](#), and includes icons by [Flaticon](#), and infographics & images by [Freepik](#)

